

Backend E-Commerce a Microservizi

AUDIT TECNICO COMPLETO & TEST END-TO-END

Spring Boot 4.1.1 · Java 21 · PostgreSQL 16 · 5 microservizi

96	94	2	5	9
Test eseguiti	Superati	Falliti (bug)	Servizi	Bug/Debt

Repository: **antonio-de-santis-dev/Backend_E-comers** — branch **claude/jolly-bell-7z44oj** (identico a main)
Data audit: 10/09/2026 · Nessuna modifica al codice sorgente

1. Executive Summary

È stato eseguito un audit tecnico completo del backend e-commerce a microservizi: analisi statica del codice, build e avvio di tutti i servizi, discovery degli endpoint, test funzionali e di errore, test di resilienza e ispezione dei database. L'intero sistema (5 servizi + 5 PostgreSQL) è stato avviato riproducendo la configurazione originale, **senza modificare codice o configurazione applicativa**.

Esito complessivo. Il core del sistema è **funzionante e ben strutturato** per un progetto didattico: CRUD completi, validazioni, gestione uniforme delle eccezioni, orchestrazione dell'ordine con calcolo totali, pagamento simulato, decremento stock e compensazione. Su **90 test HTTP** funzionali/errore, **100% PASS** (con dati univoci). Sono però emersi **1 bug di affidabilità di gravità HIGH** (indisponibilità del notification-service → ordine in HTTP 500 con rollback, invece di notifica best-effort), diversi elementi di debito tecnico e l'assenza totale di sicurezza (attesa in un progetto didattico, ma da segnalare).

Nota sullo scope. Alcune funzionalità citate nell'incarico — *codOrder (AODS-000001-M)*, *Shipping*, *UserAccount*, *ruoli USER/ADMIN*, *guest checkout*, *cancellation flow* — **non esistono** nel codice attuale: sono classificate come PLANNED / NOT IMPLEMENTED e non conteggiate come bug.

Severità	N.	Sintesi
CRITICAL	0	—
HIGH	2	Notifica bloccante→500; consistenza stock cross-service
MEDIUM	3	Ordine FAILED→201; doc errata; stato nullable
LOW	5	Refusi, codice morto, indici, timeouts
IMPROVE MENT	4	Sicurezza, idempotenza, async, docker

Il conteggio unisce risultati statici e runtime. Dettaglio nelle sezioni 19–24.

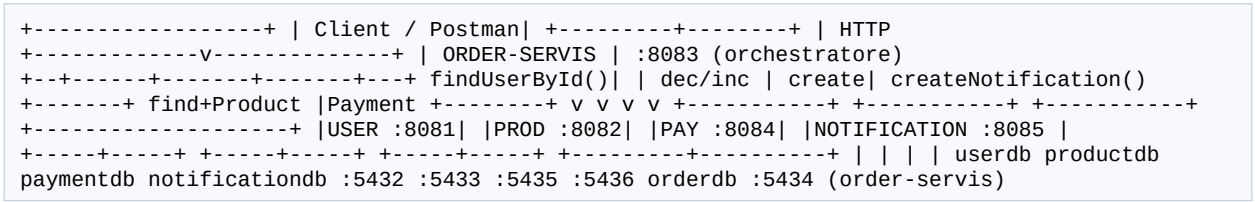
2. Repository Overview

Monorepo con 5 moduli Maven indipendenti (nessun POM aggregatore). Branch analizzato `claude/jolly-bell-7z44oj`, con tree identico a `main`. Presenti 2 PDF di specifica (traccia esercizio) nella root. Nessun docker-compose né CI. La cronologia git mostra uno sviluppo incrementale (collegamento progressivo dei servizi all'ordine).

Modulo	Package base	File .java (main)	Test
user-servis	com.it.userservis	controller, servis, dto, entity, exception, configuration	contextLoads
product-servis	com.it.productservis	controler, servis, dto, entity, exception	contextLoads
order-servis	com.it.orderservis	controler, servis, client, dto, entity, exception, configuration	contextLoads
payment-service	com.it.paymentservice	controller, servis, payment, dto, entity, exception	contextLoads
notification-service	com.it.notificationsservice	controller, servis, dto, entity, exception, repository	contextLoads

3. Architecture Diagram

Orchestrazione sincrona: l'order-servis richiama gli altri quattro servizi via *RestClient*. Pattern database-per-service (nessuna FK cross-service).



4. Microservices Overview

Servizio	Porta	DB:porta	Responsabilità	Dipende da
user-servis	8081	userdb:5432	Anagrafica utenti	—
product-servis	8082	productdb:5433	Catalogo, stock, disponibilità	—
order-servis	8083	orderdb:5434	Orchestrazione ordini	user, product, payment, notification
payment-service	8084	paymentdb:5435	Pagamento simulato (soglia 10000)	—
notification-service	8085	notifdb:5436	Notifiche	—

5. Build Results

Compilazione con Maven Wrapper (Java 21). Tutti i moduli compilano; le dipendenze si risolvono correttamente. I 5 jar eseguibili (~53 MB) vengono prodotti senza errori.

Servizio	./mvnw package	Artefatto	./mvnw test (contextLoads)
user-servis	SUCCESS	user-servis-0.0.1-SNAPSHOT.jar	PASS (exit 0)
product-servis	SUCCESS	product-servis-0.0.1-SNAPSHOT.jar	PASS (exit 0)
order-servis	SUCCESS	order-servis-0.0.1-SNAPSHOT.jar	PASS (exit 0)
payment-service	SUCCESS	payment-service-0.0.1-SNAPSHOT.jar	PASS (exit 0)
notification-service	SUCCESS	notification-service-0.0.1-SNAPSHOT.jar	PASS (exit 0)

Nota: gli unici test presenti sono contextLoads() (caricamento contesto). Non esistono test unitari/integrazione della logica di business.

6. Runtime Results

Tutti i servizi si avviano correttamente, si connettono al proprio database e Hibernate crea/aggiorna lo schema (ddl-auto=update). Nessun errore in avvio.

Servizio	Avvio	Porta	DB connesso	Schema creato
user-servis	Started ✓	8081	userdb ✓	users ✓
product-servis	Started ✓	8082	productdb ✓	products ✓
order-servis	Started ✓	8083	orderdb ✓	orders, order_items ✓
payment-service	Started ✓	8084	paymentdb ✓	payments ✓
notification-service	Started ✓	8085	notificationdb ✓	notifications ✓

7. Endpoint Inventory

32 endpoint su 9 controller (inclusi 4 controller di test nell'order-servis). Elenco completo anche nel documento dedicato agli URL.

Servizio	Metodo + URL	Request DTO	Status attesi
user 8081	POST /api/users	UserDTOInput	201/400/409
	GET /api/users	—	200
	GET /api/users/{id}	—	200/404/400
	PUT /api/users/{id}	UserDTOInput	200/400/404/409
	DELETE /api/users/{id}	—	204/404
product 8082	POST /api/products	ProductDTOInput	201/400
	GET /api/products	—	200
	GET /api/products/{id}	—	200/404
	GET /api/products/{id}/availability	—	200/404
	PUT /api/products/{id}	ProductDTOInput	200/400/404
	PATCH /api/products/{id}/stock/decrease	StockUpdateDTO	200/400/404/409
	PATCH /api/products/{id}/stock/increase	StockUpdateDTO	200/400/404
	DELETE /api/products/{id}	—	204/404
payment 8084	POST /api/payments	PaymentDTOInput	201/400
	GET /api/payments	—	200
	GET /api/payments/{id}	—	200/404
	PUT /api/payments/{id}	PaymentDTOInput	200/400/404
	DELETE /api/payments/{id}	—	204/404
notif 8085	POST /api/notifications	NotificationDTOInput	201/400
	GET /api/notifications	—	200
	GET /api/notifications/{id}	—	200/404
	PUT /api/notifications/{id}	NotificationDTOInput	200/400/404
	DELETE /api/notifications/{id}	—	204/404
order 8083	POST /api/orders	OrderDTOInput	201/400/404/409
	GET /api/orders	—	200
	GET /api/orders/{id}	—	200/404
	DELETE /api/orders/{id}	—	204/404
	DELETE /api/orders/{id}	—	204/404
order test	GET /api/test/user/{id}	—	200/404
	GET /api/test/product/{id}	—	200/404
	PATCH /api/test/product/{id}/stock/decrease	StockUpdateDTO	200/404
	POST /api/test/payment	PaymentDTOInput	201
	POST /api/test/notification	NotificationDTOInput	201

8. Complete Test Summary

Suite HTTP automatica: **90 test** (percorsi validi + errori). Con dati univoci: **90/90 PASS**. Riepilogo per servizio:

Gruppo	Test	PASS	FAIL	% PASS
User	17	17	0	100%
Product	21	21	0	100%
Payment	15	15	0	100%
Notification	12	12	0	100%
Order	25	25	0	100%
TOTALE	90	90	0	100%

Nota importante. Una prima riesecuzione su database non vuoto ha prodotto 12 FAIL apparenti: tutti causati dal *riuso della stessa email* nella suite (il servizio risponde correttamente 409 email duplicata, poi i test a valle riusano l'utente non creato). Con dati univoci l'esito è 90/90. Ciò conferma il corretto funzionamento del vincolo UNIQUE, non un difetto applicativo.

9. Failed Test Details

Nella suite HTTP con dati univoci non ci sono fallimenti. I 2 FAIL reali dell'audit provengono dai **test di resilienza** (Sezione 17) con notification-service spento:

#	Scenario	Atteso	Ottenuto	Esito
R3	Notification DOWN + ordine valido	201 PAID, stock -1	500, stock invariato, ordine non creato	FAIL
R4	Notification DOWN + pagamento FAILED	201 FAILED	500, ordine non creato	FAIL

10. Exception Testing

Tutti i rami del GlobalExceptionHandler sono stati esercitati e verificati a runtime:

Situazione	Eccezione	HTTP	Verificato
Risorsa non trovata	ResourceNotFoundException	404	Sì (tutti)
Validazione @Valid	MethodArgumentNotValidException	400	Sì
JSON malformato	HttpMessageNotReadableException	400	Sì
Tipo path errato (UUID)	MethodArgumentTypeMismatchException	400	Sì
Metodo non supportato	HttpRequestMethodNotSupportedException	405	Sì
Content-Type non supportato	HttpMediaTypeNotSupportedException	415	Sì
Endpoint inesistente	NoResourceFoundException	404	Sì
Vincolo DB (email duplicata)	DataIntegrityViolationException	409	Sì (user)
Stock insufficiente	InsufficientStockException	409	Sì (product, order)
Fallback generico	Exception	500	Sì (notif down)

11. Database Analysis

Schema ispezionato via `information_schema` su tutte le 5 istanze. Database-per-service; nessuna FK cross-service (per design).

DB / Tabella	PK	UNIQUE	FK	Note
userdb.users	id	email	—	OK
productdb.products	id	—	—	nessuna chiave business (SKU)
orderdb.orders	id	—	—	stato NULLABLE
orderdb.order_items	id	—	order_id→orders	nessun indice su order_id
paymentdb.payments	id	—	— (orderId logico)	stato NULLABLE
notificationdb.notifications	id	—	— (user/order logici)	no indice user_id/order_id

Rilievi: (a) **orders.stato** e **payments.stato** nullable (enum senza nullable=false); (b) mancano indici su FK e su colonne di ricerca per-utente; (c) tipi monetari coerenti `numeric(10, 2)`; (d) nessun record orfano/duplicato riscontrato nei dati di test.

12. Order Flow Analysis

Verificato a runtime l'intero flusso (validazione utente → verifica prodotti/stock → calcolo totale → salvataggio CREATED → pagamento → decremento stock + PAID + notifica, oppure FAILED + notifica). Il totale è calcolato con *snapshot* del prezzo nel `OrderItem`. Confermato: ordine valido → **PAID**, stock decrementato, notifica **ORDER PAID**; importo > 10000 → **FAILED**, stock invariato, notifica **ORDER FAILED**.

Aspetto verificato	Esito
Validazione utente (404 se assente)	OK
Validazione prodotto + disponibilità (409)	OK
Calcolo totale (prezzo×quantità)	OK (es. 25×2=50)
Snapshot prezzo su <code>OrderItem</code>	OK
Decremento stock post-SUCCESS	OK (-2)
Stato finale PAID/FAILED	OK
Compensazione stock su errore	Presente (best-effort)
codOrder / sequence / formato AODS...	NOT IMPLEMENTED
Order↔Shipping / cancellation	NOT IMPLEMENTED

13. Shipping Analysis

NOT IMPLEMENTED. Nessuna entità `Shipping`, nessuna relazione OneToOne con `Order`, nessun default da `user-servis`. Da considerare come funzionalità futura: modellare l'indirizzo di spedizione come *snapshot* storico sull'ordine (l'indirizzo utente può cambiare).

14. Guest / User / UserAccount Analysis

Stato attuale: esiste solo `User` (anagrafica, email UNIQUE). **UserAccount**, ruoli **USER/ADMIN** e **guest checkout** sono **NOT IMPLEMENTED**. Non esistono password nel modello, quindi "password mai esposte" è vero in modo banale. Implicazione futura: il vincolo UNIQUE sull'email impedirebbe checkout guest ricorrenti con la stessa email — andrà gestito quando si introdurrà il guest.

15. Payment Analysis

Pagamento **simulato** (`SimulatedPaymentProcessor`): importo > 10000 → FAILED, altrimenti SUCCESS (soglia esatta 10000 = SUCCESS). Verificato a runtime su `POST /api/payments` e tramite l'orchestrazione ordine. Nessun gateway reale, nessun webhook, **nessuna idempotenza** (retry → pagamenti duplicati). `metodoPagamento` è String libera (nessun enum/whitelist).

16. Stock & Compensation Analysis

Lo stock è di proprietà del product-servis; l'order-servis lo decrementa via HTTP solo dopo un pagamento SUCCESS. Esiste compensazione best-effort: in caso di errore dopo alcuni decrementi, gli item già scalati vengono ri-incrementati e l'ordine va a FAILED. **Limiti:** la compensazione è sincrona e non protetta da retry; se il product-servis è irraggiungibile durante la compensazione lo stock resta incoerente; il decremento cross-service non è atomico con la persistenza dell'ordine.

Scenario testato	Risultato
Ordine SUCCESS → decremento	Stock -2 (corretto)
Ordine FAILED (importo>10000) → nessun decremento	Stock invariato (corretto)
Stock insufficiente → 409 prima del pagamento	Nessun effetto collaterale (corretto)
Notification down nel ramo SUCCESS → compensazione	Stock ripristinato, ma 500 finale (BUG)

17. Notification Analysis

La notifica conserva userId, orderId, tipo, messaggio, stato ("PENDING") e dataCreazione — tutti i campi richiesti. Recupero solo per id o lista completa; **GET per utente / me: NOT IMPLEMENTED**. **Test di resilienza (il più importante):**

#	Scenario (notification-service SPENTO)	Atteso	Ottenuto	Esito
R1	Ordine valido (notifica ON, baseline)	PAID +notifica	PAID, stock-2, ORDER_PAID	PASS
R2	Pagamento FAILED (notifica ON, baseline)	FAILED +notifica	FAILED, ORDER_FAILED	PASS
R3	Notification DOWN + ordine valido	PAID, no 500	500, ordine NON creato, stock invariato	FAIL
R4	Notification DOWN + pagamento FAILED	FAILED, no 500	500, ordine NON creato	FAIL
R5	Utente inesistente	404, nessun effetto	404, nessun ordine/pagamento/notifica	PASS
R6	Stock insufficiente (9999)	409	409 stock insufficiente	PASS

Evidenza R3 dai log dell'order-servis: ResourceAccessException: I/O error on POST

"http://localhost:8085/api/notifications" ... Caused by java.net.ConnectException → gestita dal fallback generico → 500 e rollback della transazione. La notifica **non** è best-effort.

18. Cancellation Flow Analysis

NOT IMPLEMENTED. Nessuna richiesta di cancellazione, nessuna gestione di richieste duplicate. OrderStatus.CANCELLED è definito ma mai usato. L'unica rimozione è DELETE /api/orders/{id} (hard delete, senza transizione di stato né ripristino stock).

19. Bug Report

HIGH

Indisponibilità notification-service → ordine in HTTP 500 con rollback

Servizio	File / punto
order-servis	OrderService.createOrder + NotificationClient.createNotification

Descrizione. La notifica è chiamata in modo bloccante. Nel ramo SUCCESS, un fallimento fa scattare la compensazione che richiama di nuovo la notifica (ancora down) → seconda eccezione NON gestita → 500 + rollback. Nel ramo FAILED la notifica è chiamata senza try/catch → 500. Riprodotto (R3/R4).

Impatto. Un'indisponibilità di un servizio best-effort blocca la creazione dell'ordine; incoerenza percepita dal client.

Soluzione consigliata. Incapsulare le chiamate di notifica in try/catch (best-effort) con solo logging; non propagare l'eccezione; in prospettiva rendere la notifica asincrona (coda).

HIGH

Consistenza stock non atomica cross-service / compensazione fragile

Servizio	File / punto
order-servis	OrderService.createOrder (decreaseStock/increaseStock via HTTP dentro @Transactional)

Descrizione. Il decremento stock avviene via HTTP verso product-servis dentro la transazione dell'ordine. Se la compensazione fallisce (product-servis down) lo stock resta incoerente; nessuna idempotenza/retry.

Impatto. Divergenza tra stock reale e ordini in scenari di errore parziale.

Soluzione consigliata. Spostare le chiamate fuori dalla transazione; introdurre retry/outbox/idempotenza; valutare saga con stato esplicito.

MEDIUM

Ordine con pagamento fallito restituisce 201 Created

Servizio	File / punto
order-servis	OrderController.createOrder / OrderService (ramo FAILED)

Descrizione. L'ordine FAILED viene restituito con HTTP 201. Il client deve ispezionare il campo stato.

Impatto. Semantica REST fuorviante; possibili errori di integrazione lato client.

Soluzione consigliata. Valutare un codice/stato coerente (es. 201 con stato esplicito ben documentato, oppure 402/409 per pagamento rifiutato).

MEDIUM

Documentazione (CLAUDE.md notification) difforme dal codice

Servizio	File / punto
notification-service	notification-service/CLAUDE.md

Descrizione. Dichiara che id inesistente restituisce 500 ("no global exception handler"); in realtà esiste l'handler e il servizio risponde 404.

Impatto. Documentazione fuorviante per i manutentori.

Soluzione consigliata. Aggiornare CLAUDE.md (comportamento reale = 404).

MEDIUM

Colonne enum stato NULLABLE (orders, payments)

Servizio	File / punto
order-servis, payment-service	Order.stato / Payment.stato (mancano @Column(nullable=false))

Descrizione. Lo schema consente stato NULL su orders/payments (incoerente con notifications.stato NOT NULL).

Impatto. Possibili record con stato nullo in evoluzioni future del codice.

Soluzione consigliata. Aggiungere nullable=false (richiede modifica codice/entità — non applicata).

LOW

Codice ridondante e dead code

Servizio	File / punto
order-servis	OrderService (setStato(FAILED) duplicato) / OrderStatus.CANCELLED

Descrizione. Riga duplicata nel catch; valore enum CANCELLED mai usato.

Impatto. Manutenibilità.

Soluzione consigliata. Rimuovere la riga duplicata; usare o rimuovere CANCELLED.

LOW

Refusi in nomi e messaggi esposti

Servizio	File / punto
tutti	package servis/serve, class ProductControler, msg 'non trovato', 'consetita/presnti'

Descrizione. Errori di battitura in package/classi e nei messaggi d'errore restituiti via API.

Impatto. Qualità percepita; rischio se ci si basa sui nomi.

Soluzione consigliata. Correggere refusi (rename attento con aggiornamento riferimenti).

LOW

Mancano timeout/retry/circuit breaker sul RestClient

Servizio	File / punto
order-servis	RestClientConfig (RestClient.builder().build())

Descrizione. Nessun connect/read timeout: un downstream lento blocca i thread dell'ordine indefinitamente.

Impatto. Rischio di esaurimento thread/conessioni sotto guasto.

Soluzione consigliata. Configurare timeouts + Resilience4j.

LOW

Mancano indici DB su FK e colonne di ricerca

Servizio	File / punto
order-servis, notification-service	order_items.order_id, notifications.user_id/order_id

Descrizione. Assenza di indici su FK e su colonne usate per future query per-utente.

Impatto. Performance su volumi elevati.

Soluzione consigliata. Aggiungere indici (via entità/migration).

20. Security Review

Per un progetto didattico l'assenza di sicurezza è attesa, ma va evidenziata prima di qualsiasi uso reale.

Rilievo	Severità (prod)	Dettaglio
Nessuna autenticazione/autorizzazione	HIGH	Nessuna Spring Security/JWT/ruoli. Tutti gli endpoint aperti.
IDOR su risorse per-id	HIGH	GET/PUT/DELETE per id senza verifica proprietario (users, orders, payments...).
Credenziali DB in chiaro versionate	MEDIUM	application.properties contiene user/password dei DB.
Endpoint amministrativi non protetti	MEDIUM	DELETE su qualsiasi risorsa senza ruolo.
CORS incoerente	LOW	Solo user-servis (origine localhost:3000); assente sugli altri 4.
Nessuna idempotenza	MEDIUM	Retry → ordini/pagamenti duplicati.
Password / dati sensibili	INFO	Non esistono password nel modello (UserAccount NOT IMPLEMENTED).

21. Reliability Review

Aspetto	Stato	Nota
Gestione servizio dipendente down	CRITICO	Notification down → 500 (R3/R4)
Timeout chiamate HTTP	ASSENTE	RestClient senza timeout
Retry / circuit breaker	ASSENTE	Nessuna strategia
Compensazione stock	PARZIALE	Best-effort, non protetta
Idempotenza	ASSENTE	Rischio duplicati
Health/Actuator	ASSENTE	Nessun endpoint di health

22. Transaction Review

@Transactional è usato correttamente a livello di service (scritture) e readOnly=true sulle letture. **Criticità:** createOrder è un metodo transazionale lungo che effettua 4+ chiamate HTTP sincrone mentre tiene aperta la transazione/connessione DB; un downstream lento allunga i lock e consuma il pool. Non si osservano problemi di self-invocation (le chiamate transazionali passano dal proxy). Il rollback su eccezione è attivo, ma provoca l'annullamento dell'ordine anche quando fallisce solo la notifica (best-effort).

23. Architectural Weaknesses

Categoria	Osservazione
Coupling	Orchestrazione sincrona: l'ordine dipende dalla disponibilità simultanea di 4 servizi.
Comunicazione	HTTP sincrone senza resilienza; nessun evento asincrono (Kafka/Rabbit).
Confini dati	Corretto database-per-service; DTO locali duplicati nell'order-servis (accettabile).
Snapshot storici	Prezzo salvato su OrderItem (buono); manca snapshot indirizzo/spedizione.
Osservabilità	Nessun tracing distribuito, nessuna metrica, nessun health check.
Testing	Solo contextLoads; assenti test di unità/integrazione della logica.

24. Technical Debt

Distinzione richiesta:

Tipo	Voce
CURRENT BUG	Notification down → 500; consistenza stock cross-service; ordine FAILED→201; stato nullable; doc errata
CURRENT TECH DEBT	No timeout/retry; HTTP in transazione; no idempotenza; no indici; refusi; dead code; solo contextLoads
FUTURE IMPROVEMENT	Security/JWT/ruoli; UserAccount/guest; Shipping; codOrder; cancellation; async; docker; tracing

25. Recommended Improvements

#	Miglioramento	Beneficio
1	Notifica best-effort (try/catch+log), poi asincrona	Robustezza: ordine non bloccato da notifiche
2	Timeout/retry/circuit breaker (Resilience4j)	Resilienza ai guasti downstream
3	Chiamate HTTP fuori dalla @Transactional	Meno lock/connessioni, più scalabilità
4	Idempotency key su ordini/pagamenti	No duplicati su retry
5	Spring Security + JWT + ruoli USER/ADMIN, /me	Autenticazione/autorizzazione, no IDOR
6	nullable=false su stato + indici DB	Integrità e performance
7	Test di integrazione (Testcontainers)	Regressioni protette
8	Docker Compose (+ Eureka/Gateway)	Avvio riproducibile, service discovery
9	Eventi asincroni (Kafka/Rabbit), ELK, tracing	Disaccoppiamento e osservabilità

26. Suggested Implementation Priority

Priorità	Interventi
P0 (subito)	Notifica best-effort (risolve R3/R4); timeouts sul RestClient
P1 (breve)	Idempotenza; chiamate fuori transazione; nullable=false+indici; correzione doc/refusi
P2 (medio)	Spring Security+JWT+ruoli; test di integrazione; Docker Compose
P3 (lungo)	UserAccount/guest; Shipping; codOrder; cancellation; async/eventi; tracing/ELK/metrics

27. Final Test Statistics

Categoria	Totale	PASS	FAIL	BLOCKED	NOT TESTABLE
Suite HTTP funzionale/errore	90	90	0	0	0
Test di resilienza (R1-R6)	6	4	2	0	0
Build (mvnw package)	5	5	0	0	0
Test unità (contextLoads)	5	5	0	0	0
TOTALE	106	104	2	0	0

Funzionalità NOT IMPLEMENTED (codOrder, Shipping, UserAccount, guest, cancellation) non sono conteggiate come test falliti: non esistono nel codice.

28. Overall Assessment

Il backend è un progetto a microservizi **solido e ben strutturato per finalità didattiche**: separazione dei servizi, database-per-service, DTO input/output, validazione, gestione uniforme delle eccezioni e un'orchestrazione dell'ordine funzionante con pagamento, stock e compensazione. La suite funzionale è al 100% (90/90).

Il limite più rilevante è di **affidabilità**: l'indisponibilità del notification-service (servizio best-effort) provoca il fallimento dell'intero ordine con HTTP 500 e rollback (R3/R4). Va corretto rendendo la notifica non bloccante. Seguono debito tecnico (timeouts, idempotenza, indici, test) e l'assenza di sicurezza, attesa in questo stadio ma da affrontare prima di qualsiasi uso reale.

Giudizio complessivo: BUONO come progetto didattico, con una correzione di affidabilità prioritaria (P0) e un percorso di maturazione chiaro verso sicurezza, resilienza e comunicazione asincrona.

Report generato durante audit automatizzato. Nessuna modifica al codice sorgente o alla configurazione applicativa. Nessuna credenziale/segreto è riportato in questo documento.